7.Implementation of Random Forest and Performance Analysis

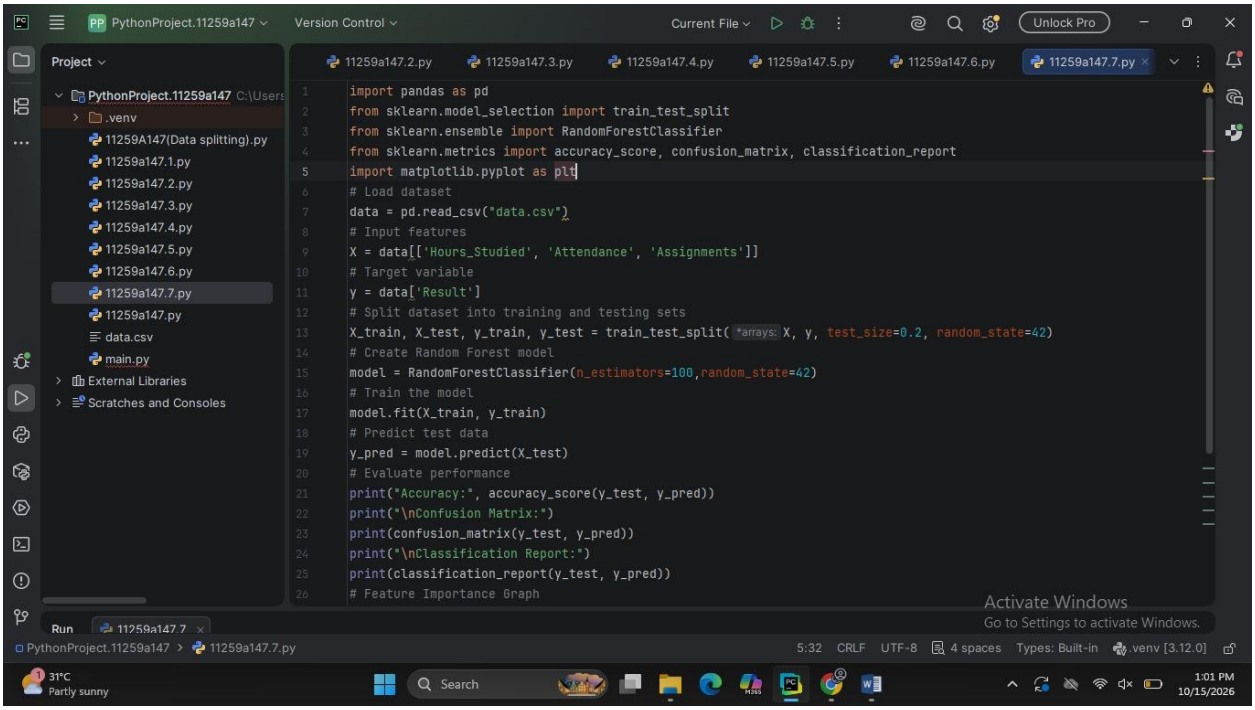
Aim:

To write a program that implements a Random Forest classifier, which combines many decision trees to make predictions, trains it on a training set, predicts the class labels for the test set, and analyzes its performance using accuracy, the confusion matrix, the classification report and feature importance.

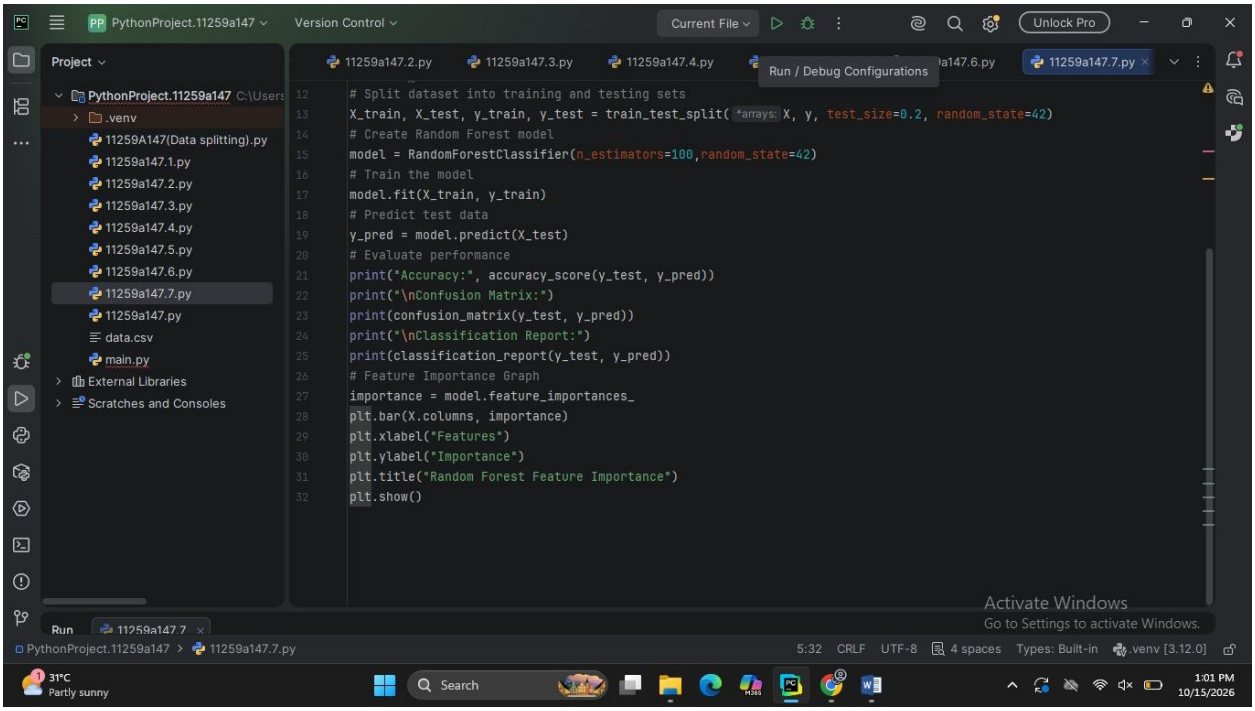
Algorithm:

1. Start the program.
2. Import the required libraries (pandas, matplotlib, train_test_split from sklearn.model_selection, RandomForestClassifier from sklearn.ensemble, and accuracy_score, confusion_matrix and classification_report from sklearn.metrics).
3. Load the dataset from the input file.
4. Separate the data into input features (X) and the target variable (y).
5. Split the data into training and testing sets using train_test_split (for example 80% training and 20 % testing), with a fixed random_state.
6. Create the RandomForestClassifier model with a chosen number of trees n_estimators (for example 100) and a fixed random_state.
7. Train the model by fitting it on the training data (X_train, y_train).
8. The model builds many decision trees, each on a random sample of the training data and a random subset of the features.
9. Predict the class labels for the test set (X_test); each tree votes and the majority class is chosen as the final prediction.
10. Compare the predicted labels with the actual labels (y_test) and calculate the accuracy.
11. Build the confusion matrix and the classification report (precision, recall and F1-score).
12. Find the importance of each feature using feature_importances_ and display it (optional: plot it as a bar chart).
13. Display the evaluation results and analyze the performance of the model.
14. Stop the program.

Program:

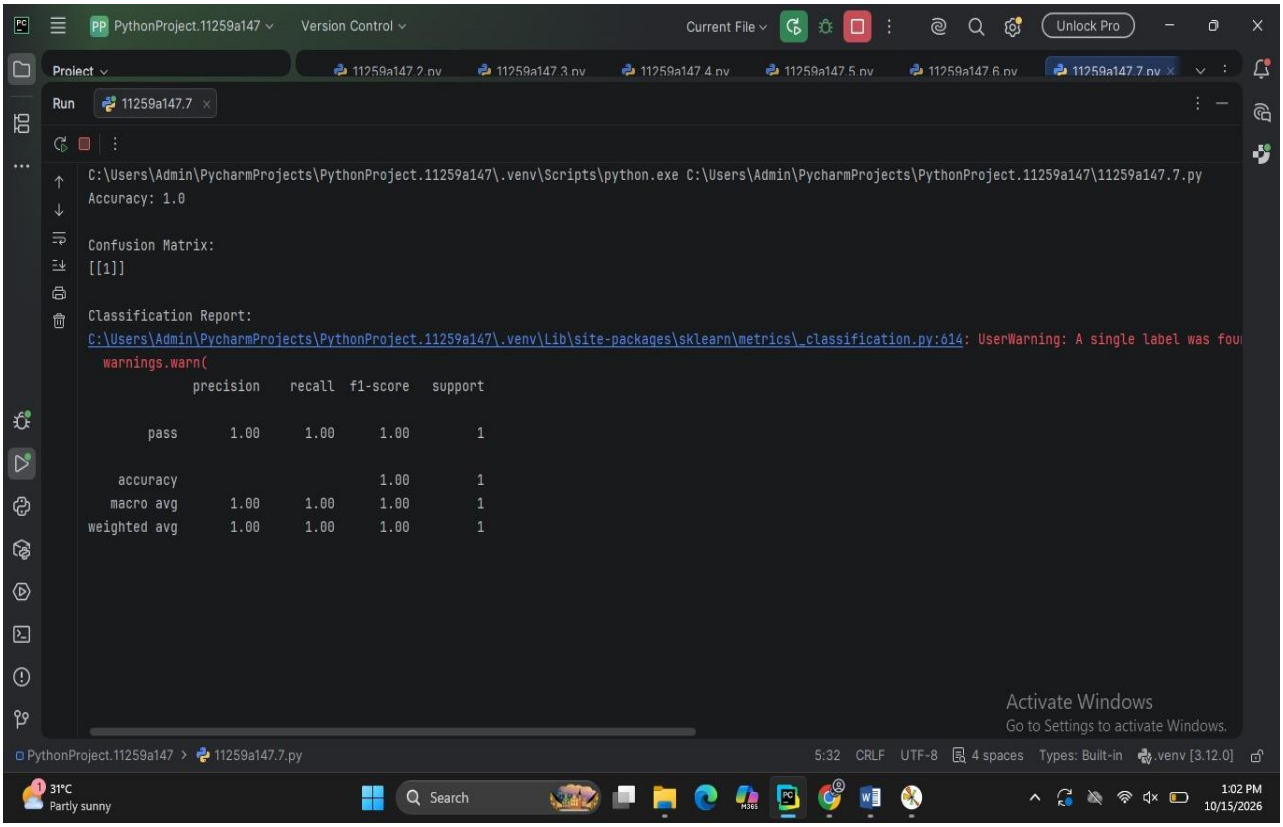


```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
# Load dataset
data = pd.read_csv("data.csv")
# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]
# Target variable
y = data['Result']
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)
# Create Random Forest model
model = RandomForestClassifier(n_estimators=100,random_state=42)
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Evaluate performance
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# Feature Importance Graph
```

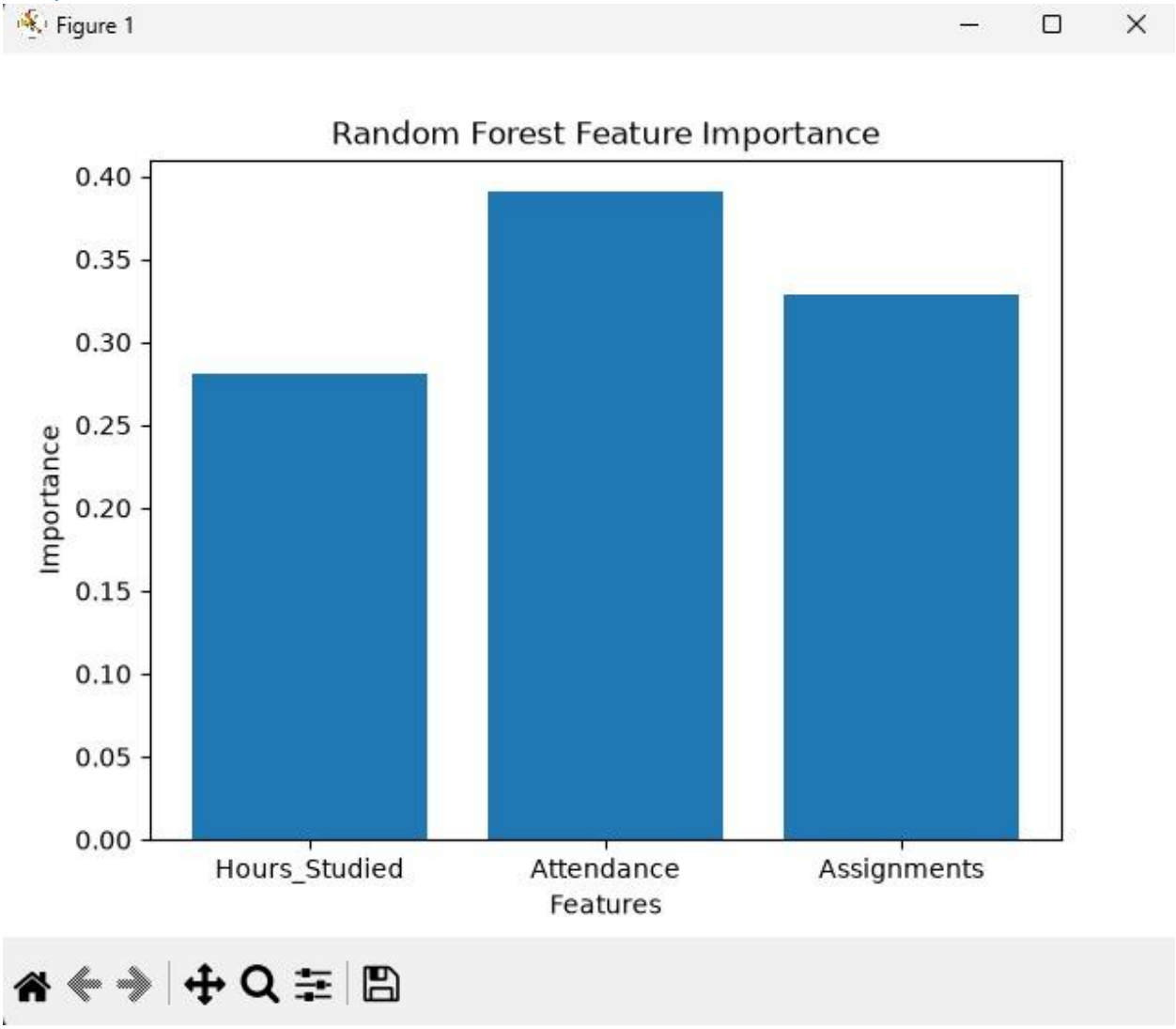


```
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)
# Create Random Forest model
model = RandomForestClassifier(n_estimators=100,random_state=42)
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Evaluate performance
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# Feature Importance Graph
importance = model.feature_importances_
plt.bar(X.columns, importance)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Random Forest Feature Importance")
plt.show()
```

Result:


PythonProject.11259a147
Version Control
Current File
Unlock Pro
Project
11259a147.7.py
Run
11259a147.7
C:\Users\Admin\PycharmProjects\PythonProject.11259a147\.venv\Scripts\python.exe C:\Users\Admin\PycharmProjects\PythonProject.11259a147\11259a147.7.py
Accuracy: 1.0
Confusion Matrix:
[[1]]
Classification Report:
C:\Users\Admin\PycharmProjects\PythonProject.11259a147\.venv\Lib\site-packages\sklearn\metrics_classification.py:614: UserWarning: A single label was fou
warnings.warn(
precision recall f1-score support
pass 1.00 1.00 1.00 1
accuracy 1.00 1
macro avg 1.00 1.00 1.00 1
weighted avg 1.00 1.00 1.00 1
Activate Windows
Go to Settings to activate Windows.
PythonProject.11259a147 > 11259a147.7.py
5:32 CRLF UTF-8 4 spaces Types: Built-in .venv [3.12.0]
31°C
Partly sunny
Search
1:02 PM
10/15/2026

Graph:


Figure 1
Random Forest Feature Importance
Importance
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Hours_Studied
Attendance
Assignments
Features